

3. Run the `migrate` command: create tables in a database based on the migration file

As we haven't touched `models.py`, we'll skip the first two steps in this topic. We'll cover those two steps in detail later.

Just for checking purposes, run the `makemigration` command. You'll see that nothing happens.

Command Line - INPUT

```
(d_env) project_d % | python manage.py  
makemigrations
```

Command Line - RESPONSE

```
No changes detected
```

However, there are several migration files already prepared for basic Django features. By running the `migrate` command, Django creates tables in a database using the preset migration files. The migration command is executed using the `manage.py` file like the shown below.

Command Line - INPUT

```
(d_env) project_d % | python manage.py  
migrate
```

When you run the command above, you'll see that migrations are applied to several migration files.

Command Line - RESPONSE

```
Operations to perform:  
Apply all migrations: admin, auth, contenttypes, sessions  
Running migrations:  
Applying contenttypes.0001_initial... OK  
Applying auth.0001_initial... OK  
Applying admin.0001_initial... OK  
Applying admin.0002_logentry_remove_auto_add... OK  
Applying admin.0003_logentry_add_action_flag_choices... OK  
Applying contenttypes.0002_remove_content_type_name... OK  
Applying auth.0002_alter_permission_name_max_length... OK  
Applying auth.0003_alter_user_email_max_length... OK  
Applying auth.0004_alter_user_username_opts... OK  
Applying auth.0005_alter_user_last_login_null... OK  
Applying auth.0006_require_contenttypes_0002... OK  
Applying auth.0007_alter_validators_add_error_messages... OK
```